Declarative vs Imperative Approach in Terraform

Introduction

Imagine you need to build a house.

- **Would you prefer to describe the final design and let the construction team handle the details?**
- **Or would you rather instruct them step by step on how to place every brick and mix the cement?**

These two methods represent the **Declarative and Imperative** approaches in Terraform and Infrastructure as Code (IaC).

Section 1: Learn

What is the Declarative Approach?

The declarative approach describes the desired state of the infrastructure, and Terraform ensures it matches that state.

- Users **specify what they want**, not how to get there.
- Terraform **determines the steps required** to reach the desired state.
- Example: **Terraform configuration files** define the final infrastructure without explicit commands.

What is the Imperative Approach?

The imperative approach provides step-by-step instructions to achieve the desired infrastructure.

- Users **specify how** to achieve the result.
- Every step must be **manually defined and executed in order**.

- Example: **AWS CLI or Bash scripts** where commands must be executed sequentially.

Key Differences

Feature	Declarative Approach	Imperative Approach
Definition	Describes the final state	Describes step-by-step changes
Focus	**What** should be done	**How** it should be done
Example Tool	Terraform, Kubernetes	AWS CLI, Bash Scripts
Scalability	Easily scales with configurations	Requires more effort to scale
Error Handling	Auto-adjusts to maintain state	Requires manual correction
Learning Curve	Easier for long-term use	More control but harder to maintain

Why is the Declarative Approach Preferred for Cloud Infrastructure?

1. **Less Human Error** – Terraform automatically calculates necessary changes.

   2. **Easier Maintenance** – The system ensures the desired state is always achieved.

   3. **Better Scalability** – Works efficiently even with large-scale deployments.

An Interesting Anecdote

Before Terraform, IT teams had to manually execute **hundreds of commands** to set up cloud resources. With Terraform's **declarative** approach, they now **define a single configuration file, and Terraform does the rest!**

Section 2: Practice

1. Using a Declarative Approach (Terraform Example)

Create a file main.tf to define an AWS EC2 instance:

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "my_server" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
}
```

Run Terraform commands:

```
terraform init
terraform apply
```

What happens here?

- Terraform ensures the **EC2 instance is created as per the configuration**.

---

2. Using an Imperative Approach (AWS CLI Example)

Manually create the same EC2 instance step by step:

```
aws ec2 run-instances --image-id ami-12345678 --instance-type t2.micro
--count 1
```

What happens here?

- The user has to **run multiple commands** to ensure proper configuration.

Real-World Example

A **software company** needs to:

1. **Create a scalable cloud infrastructure.**
2. **Ensure all teams use the same configuration.**
3. **Easily manage changes over time.**

What did they do?

1. Used **Terraform (Declarative)** to define all resources in code.
2. Applied Terraform scripts across **different environments (dev, test, prod).**
3. Automated deployments using **CI/CD pipelines**.

Result? The company **saved time, reduced errors, and improved infrastructure consistency!**

Section 3: Know More

Frequently Asked Questions

1. **Which approach is better: Declarative or Imperative?**
    - The **Declarative approach is better** for automation and large-scale infrastructure.

- The **Imperative approach is better** when fine-grained control is needed.

2. **Does Terraform support the Imperative approach?**
    - No, **Terraform is fully declarative**.
3. **Can both approaches be used together?**
    - Yes, some cloud environments use a **mix of declarative and imperative methods**.
4. **What are other examples of declarative tools?**
    - **Kubernetes (for container orchestration)** and **Ansible (for configuration management)**.
5. **Where can I practice Terraform?**
    - Use **AWS Free Tier** and experiment with declarative configurations.

Conclusion

Understanding **Declarative vs Imperative** approaches helps in choosing the right tool for cloud infrastructure.

Start by **writing Terraform configurations, comparing them with AWS CLI commands, and automating deployments**, and soon you'll be **deploying cloud resources like an expert!**